KPR Institute of Engineering and Technology

Department of Artificial Intelligence and Data Science

U21ADP05 – Exploratory Data Analysis and Visualization | Assignment II – Case Study

Exploratory Data Analysis and Visualization of Olist Customer Reviews

Text Data: Sentiment Analysis of Portuguese E-Commerce Reviews with a Deep Neural Network

Field	Details
Student Name	RYAN TITUS ABISHEK
Roll No.	23AM058
Department	AIML
Year / Semester	IV Year / VII Sem
Batch	2023–2027
Course Coordinator	Dr. Karthick Paneerselvam
Date of Submission	15/10/2026

Abstract

Online marketplaces depend on customer reviews, but free-text comments are hard to analyse at scale. This study explores the Olist Brazilian e-commerce dataset, merging nine tables into one review-level table of 44 fields and 40,642 Portuguese comments. After cleaning, imputation, outlier treatment and a stratified 70/15/15 split, seven-stage visualization and statistical tests showed that late delivery, long comments and product category are linked to negative sentiment. A multi-layer perceptron on TF-IDF features, tuned over eight configurations, classified comments as negative, neutral or positive with 82.1% test accuracy, macro-F1 0.684 and macro AUC 0.895, outperforming a logistic-regression baseline. The neutral class remains hardest to predict.

(104 words)

1. Introduction and Objective

Customer reviews are a primary signal of product quality and logistics performance for online marketplaces. Olist connects small Brazilian businesses to large marketplaces; after a purchase, customers rate the order from 1 to 5 stars and may add a free-text comment. Reading every comment manually is impractical, so automatic understanding of review text is valuable for customer-service triage, seller feedback and quality monitoring.

Objectives

- Acquire and merge a real-world multi-table dataset and extract a text-based problem from it (≥15 fields).
- Perform EDA and preprocessing: missing values, duplicates, outliers, encoding, scaling, and a train/validation/test split.
- Create at least five meaningful visualizations following the seven stages of visualization, with univariate, bivariate, multivariate and distribution-based analysis.
- Build and tune a deep neural network for text classification and evaluate it with loss/accuracy curves, a confusion matrix, ROC/AUC and error analysis.
- Draw practical conclusions and recommendations for the marketplace.

Mapping to the seven stages of visualization (Fry, 2008): Acquire (Section 2), Parse (Section 2 – merging/dates), Filter (Section 3 – cleaning), Mine (Section 3/4 – statistical tests), Represent and Refine (Section 4 – charts with consistent labels and colour code), Interact (optional Plotly chart in the notebook).

2. Dataset Description

Source: Brazilian E-Commerce Public Dataset by Olist, Kaggle – https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce (≈100k orders placed between 2016 and 2018). Eight relational CSV tables were used:

Table	Rows	Columns
reviews	99,224	7
orders	99,441	8

customers	99,441	5
items	112,650	7
payments	103,886	5
products	32,951	9
sellers	3,095	4
translation	71	2

Reviews, orders, customers, order items, payments, products (with English category names) and sellers were joined on their keys; items and payments were aggregated to one row per order. Of 99,224 reviews, 40,977 contain a written comment (the text corpus); after de-duplication and removing 1–2-character comments, 40,642 remain. The merged table has 44 columns, of which the following were used:

Field(s)	Type	Description
text / review_comment_message	Text	Portuguese customer comment (model input)
review_score → sentiment	Target	1–2 negative, 3 neutral, 4–5 positive
review_comment_title, has_title	Text / binary	Optional comment title (76% empty) and indicator
review_word_count, review_char_count	Numeric	Length of the comment
n_exclaim, n_question, upper_ratio	Numeric	Punctuation and capital-letter style features
delivery_days	Numeric	Purchase → delivery to customer (days)
delay_days	Numeric	Delivery minus estimated date (>0 = late)
approval_hours	Numeric	Purchase → payment approval (hours)
total_price, total_freight, payment_value	Numeric	Item price, freight cost and total paid (BRL)
payment_installments	Numeric (discrete)	Number of instalments
n_items, n_sellers	Numeric (discrete)	Items / sellers in the order
product_photos_qty, product_weight_g, product_volume_cm3, product_name_lenght, product_description_lenght	Numeric	Product listing attributes
product_category_name_english	Categorical	≈70 product categories
payment_type, order_status	Categorical	Payment method; order status
customer_state, seller_state	Categorical	Brazilian state codes
order_purchase_timestamp, purchase_month, purchase_hour	Date / numeric	Purchase time information

Class balance of the target: negative 10,871 (26.75%), neutral 3,533 (8.69%), positive 26,238 (64.56%). The data are imbalanced, with the neutral class under 9%, which shapes the modelling choices below.

Basic statistics (after winsorising at the 1st/99th percentile):

Feature	Mean	Std	Min	Median	Max
total_price	140.50	169.91	12.00	89.90	1098.00
total_freight	23.55	17.91	7.39	17.72	115.54
payment_installments	3.06	2.75	1.00	2.00	10.00
product_photos_qty	2.22	1.64	1.00	2.00	8.00
delivery_days	13.20	9.36	1.86	10.77	50.25
delay_days	-10.84	9.75	-35.58	-11.61	23.08
review_word_count	11.75	9.50	1.00	9.00	37.00

3. EDA and Preprocessing

3.1 Methods used

- Missing values: the comment title is missing in 76% of reviews, so it is kept only as the indicator has_title. Delivery fields are missing for 1,874 undelivered orders, product attributes for ≈1–3% of rows. Numeric gaps were filled with the median (robust to skew); missing category/payment type became the level 'unknown'.
- Duplicates: 14 repeated (order, comment) pairs were removed; comments shorter than 3 characters were dropped.
- Outliers: detected with the 1.5×IQR rule (e.g., total_price 3,420, total_freight 3,954, payment_value 3,301, payment_installments 2,911 flagged rows) and treated by winsorising at the 1st/99th percentile, because extreme prices and delivery times are genuine but would distort scaling-based methods.
- Transformation: log transforms for skewed variables (price, delivery time, comment length) in the distribution analysis.
- Encoding and scaling: one-hot encoding of 5 categorical fields (127 dummy columns); z-score standardisation of 18 numeric features (fitted on the training set only to avoid leakage).
- Text processing: lower-casing, URL removal, accent-insensitive TF-IDF on unigrams+bigrams (min_df=3, up to 20,000 features; 17,451 retained) and a Portuguese stop-word list for visual analysis.
- Split: stratified train/validation/test = 28,449 / 6,096 / 6,097 (70/15/15). Only the training set was oversampled (to 45,914 rows) so that minority classes reach 75% of the majority.

3.2 Insights from cleaning and statistical testing

- Late deliveries are strongly associated with sentiment: 26.37% of negative-review orders arrived after the estimated date versus 11.94% (neutral) and 3.68% (positive); Kruskal–Wallis $H = 1736$, $p < 0.001$.
- Comment length differs by sentiment (median 15 / 12 / 7 words for negative / neutral / positive; Kruskal–Wallis $H = 6009$, $p < 0.001$).
- Product category and sentiment are dependent ($\chi^2 = 755$, dof = 28, $p < 0.001$) but the effect is small (Cramér's $V = 0.109$); customer state is weaker still ($V = 0.062$) and payment type negligible ($V = 0.017$). With ≈40k rows even tiny effects are statistically significant, so effect size matters more than p-values.
- Spearman correlations with the 1–5 score: comment length $\rho = -0.38$, delivery time $\rho = -0.25$, delay vs estimate $\rho = -0.21$.

4. Data Visualization

All charts use a consistent colour code (red = negative, grey = neutral, green = positive), titled axes with units, and legends. Eleven figure groups (more than the required five) cover text, univariate, bivariate, multivariate, distribution and time views.

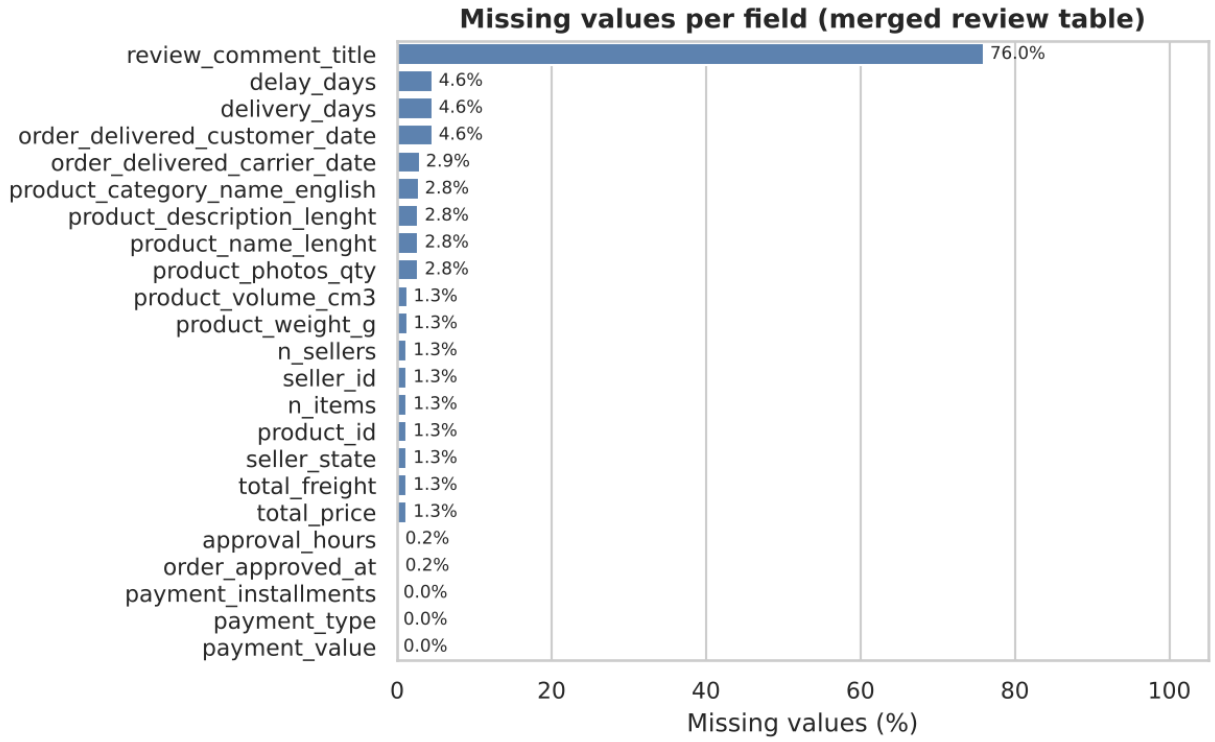


Figure 1. Percentage of missing values per field.

What it shows: The share of missing entries in each field of the merged table (title is the most incomplete).

Why it was created: To decide which fields can be imputed and which should be reduced to an indicator (Filter stage).

Insights: Comment titles are 76% empty, delivery fields 4.6% (undelivered orders) and product attributes 1–3%. Because gaps beyond the title are small, median imputation is safe.

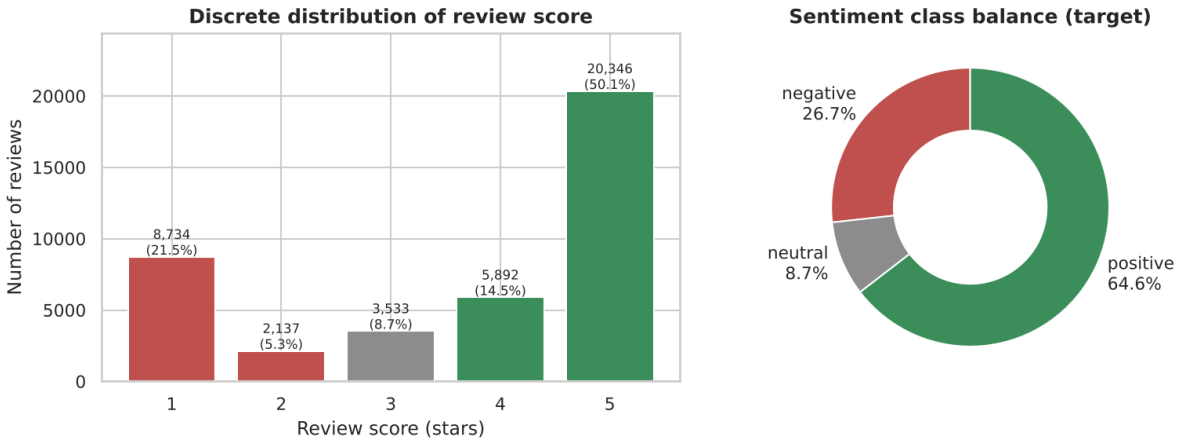


Figure 2. Distribution of review scores and the three sentiment classes.

What it shows: The discrete distribution of star ratings among reviews that include a comment, and the resulting three-class target.

Why it was created: To understand class balance before modelling, since it determines which metrics are meaningful.

Insights: 5-star reviews are the largest group (50.1%) followed by 1-star (21.5%), a U/J-shape typical of reviews: people rate at the extremes. Positive is 64.56%, negative 26.75% and neutral only 8.69%, so accuracy alone would be misleading and macro-F1 is reported.

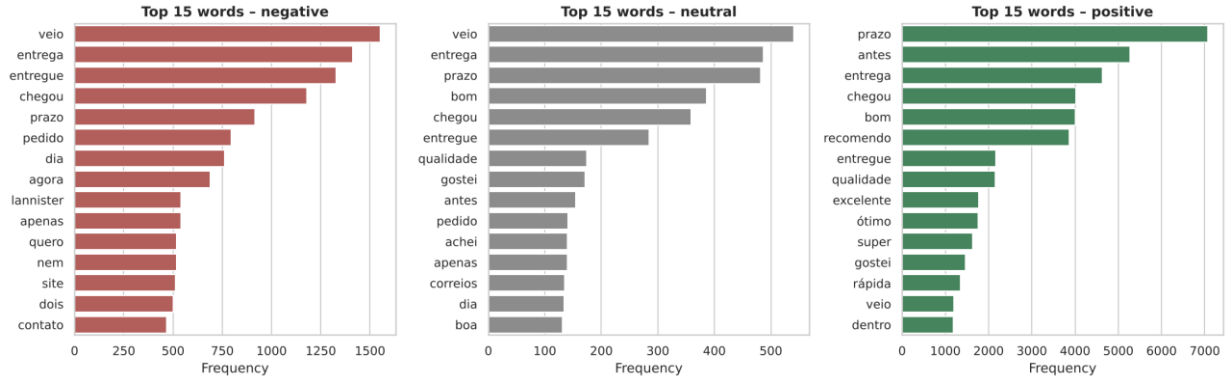


Figure 3. Top-15 words for each sentiment class (frequency plot).

What it shows: The most frequent content words in negative, neutral and positive comments after stop-word removal.

Why it was created: A basic text-mining view to see what customers talk about (Mine/Represent stages).

Insights: Delivery vocabulary (entrega, prazo, chegou, entregue) dominates every class, so logistics is the main topic of conversation. Positive comments add praise terms (recomendo, excelente, ótimo) and 'antes' (before the deadline); negative comments add 'veio' (it came [wrong/defective]), 'pedido' and 'agora' and the store name 'lannister'. Shared words across classes show that single words alone are ambiguous – context (n-grams) is needed.

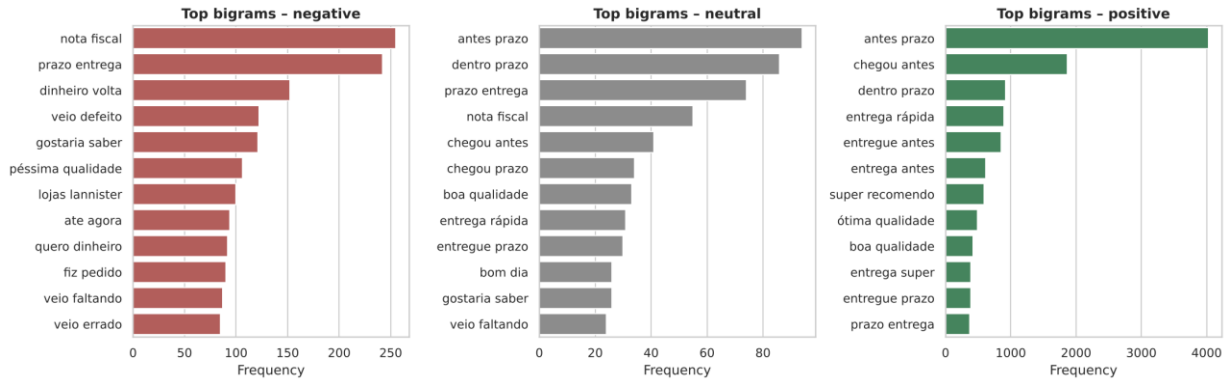


Figure 4. Top bigrams for each sentiment class.

What it shows: The most frequent two-word phrases per class.

Why it was created: Bigrams capture negation and phrases that single words miss (N-gram visualization).

Insights: Negative: 'nota fiscal' (invoice), 'veio defeito' (arrived defective), 'péssima qualidade', 'dinheiro volta' (refund), 'gostaria saber' (asking for information) – complaints about defects, invoices and refunds. Positive/neutral: 'antes prazo', 'chegou antes', 'entrega rápida', 'super recomendo', 'ótima qualidade' – speed of delivery and quality. Operational themes appear that a star rating alone would hide, such as missing invoices.

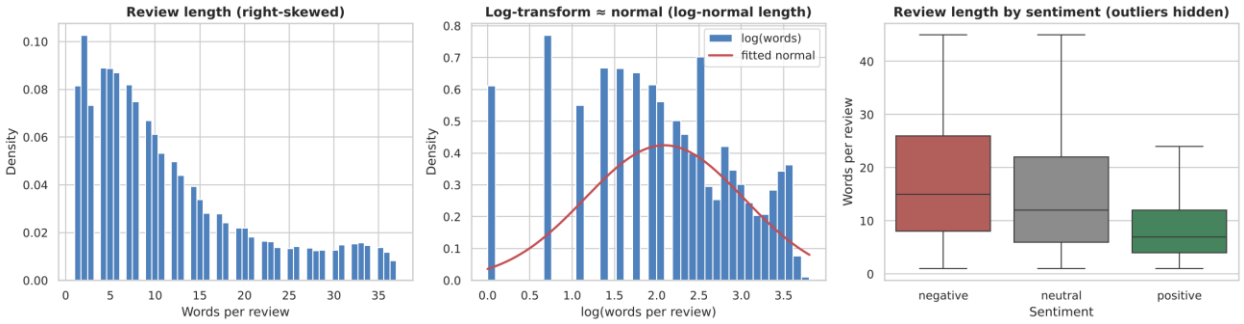


Figure 5. Review length: raw distribution, log-transformed distribution with fitted normal, and by sentiment.

What it shows: A histogram of words per comment, its log-transform with a fitted normal curve, and box plots of length by sentiment.

Why it was created: Univariate distribution analysis of a key derived text feature and its relation to the target.

Insights: Length is right-skewed (skewness 1.10); on the log scale it is much closer to symmetric (skewness -0.45), supporting a log-normal-like model. Unhappy customers write more: median 15 words (negative) versus 7 (positive).



Figure 6. Delivery delay and delivery time versus sentiment.

What it shows: Box plots of delay relative to the promised date, the percentage of late orders per star score, and violin plots of delivery time by sentiment.

Why it was created: Bivariate analysis of the operational factor that the text suggested matters most.

Insights: Negative reviews have the heaviest late tail: the upper quartile of their orders is already at the estimated date (≈0 days) while for positive orders it is about 7 days early; mean delay is -6.2 days (negative) versus -12.71 days (positive). The share of late orders falls steadily with rating – from 28.5% at 1 star to 3.1% at 5 stars – and median delivery time rises from 9.85 days (positive) to 12.23 days (negative). Reliability of the delivery promise is a clear driver of satisfaction.

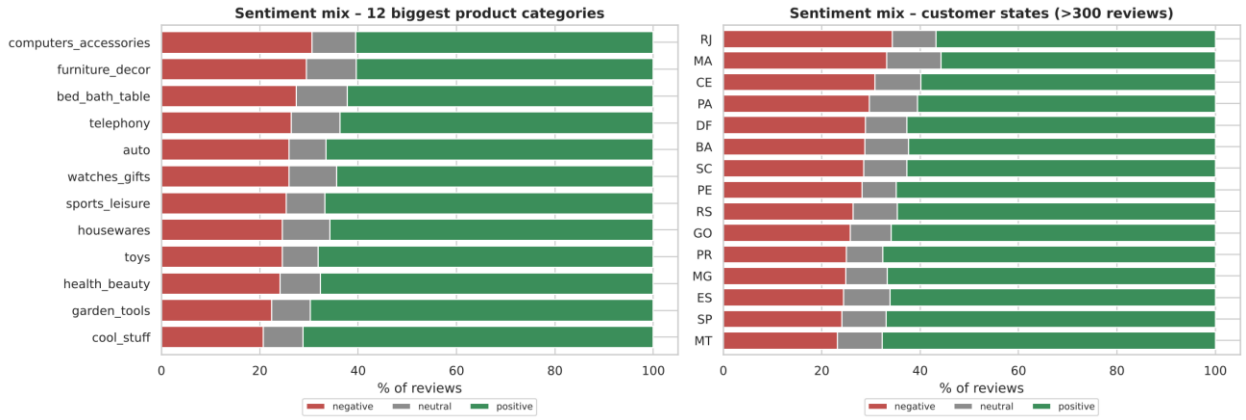


Figure 7. Sentiment mix by product category and customer state.

What it shows: 100% stacked bars of sentiment for the 12 largest categories and the states with more than 300 reviews.

Why it was created: Categorical bivariate analysis to find where dissatisfaction concentrates.

Insights: Negative share ranges from 20.7% (cool_stuff) to 30.6% (computers_accessories), and by state from 23.2% (MT) to 34.3% (RJ). Hardware/furniture categories, which are harder to ship and match to expectations, and states such as RJ, MA and CE show more complaints. The differences are statistically significant but modest (Cramér's V ≈ 0.11 and 0.06).

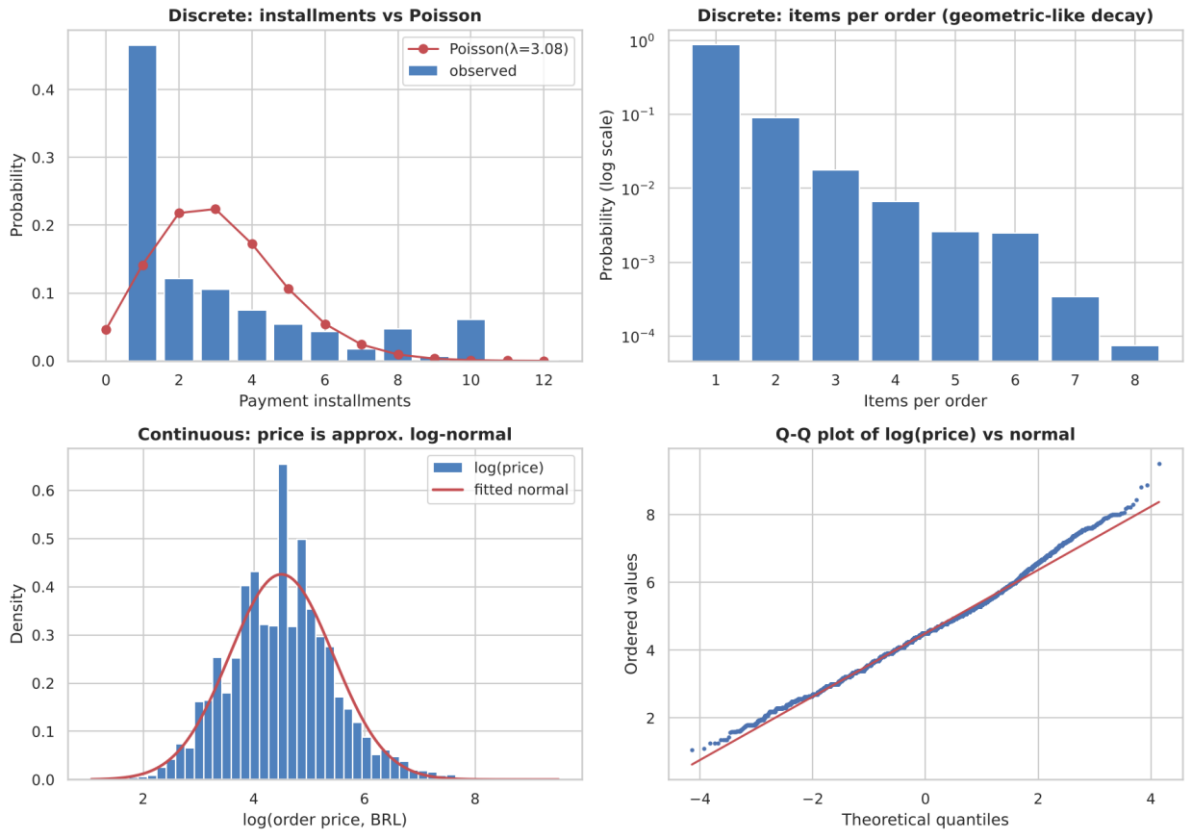


Figure 8. Discrete and continuous distribution analysis.

What it shows: Top row: observed distribution of payment instalments against a Poisson model, and items per order on a log axis. Bottom row: log-price histogram with fitted normal and a Q-Q plot.

Why it was created: To identify, visualise and test discrete and continuous probability distributions (CO4).

Insights: Instalments are not Poisson: the mean is 3.08 but the variance 7.93 (dispersion index 2.57 ≫ 1, i.e. over-dispersed), with a spike at 1 (single payment) and peaks at 8 and 10 reflecting standard instalment plans. Items per order decays roughly geometrically (most orders have one item). Order price is extremely right-skewed (skewness 11.1) but after a log transform skewness falls to 0.25 and the Q-Q plot is nearly linear, i.e. approximately log-normal, with slightly heavier tails (KS distance 0.034). Log transforms are therefore appropriate for price-like features.

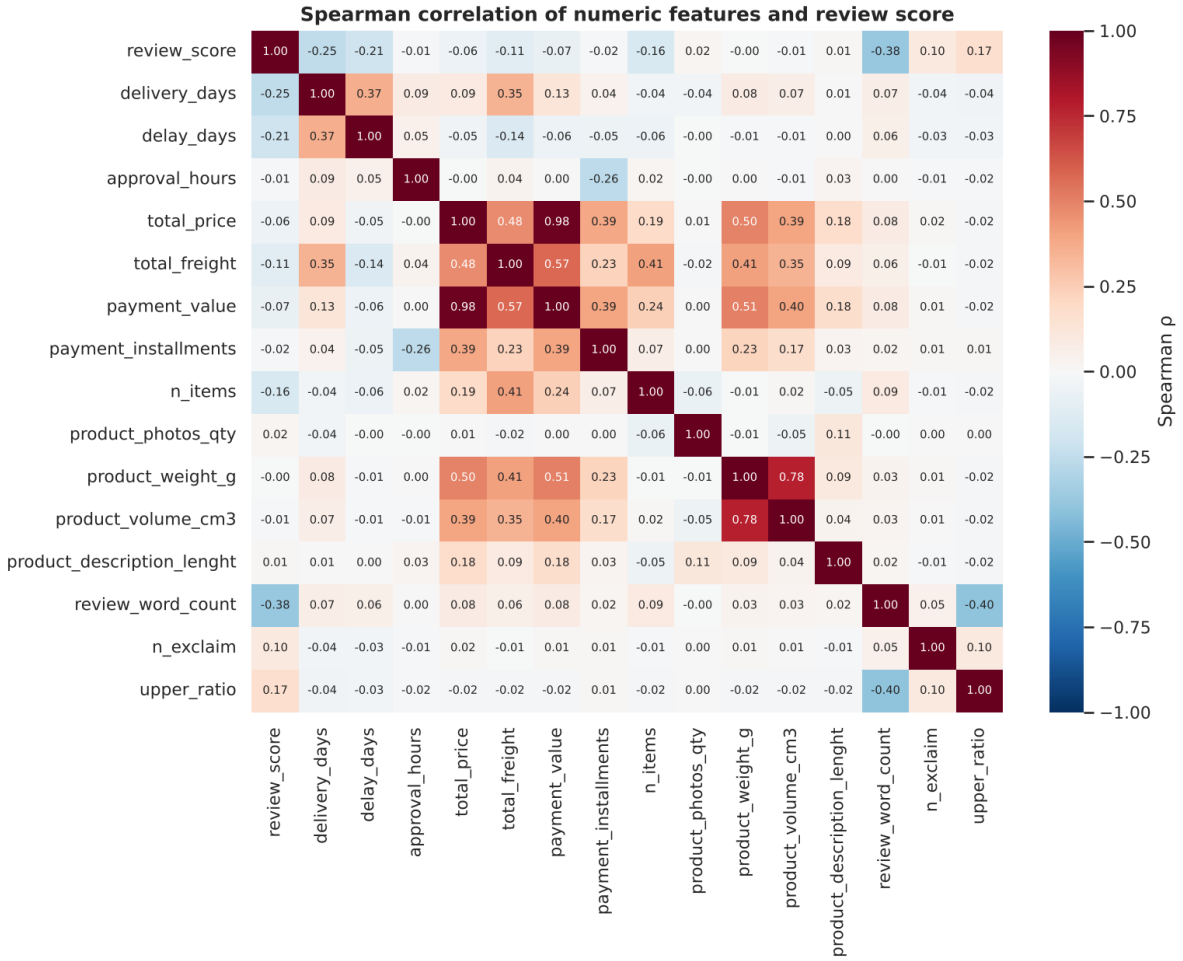


Figure 9. Spearman correlation heat map of numeric features and review score.

What it shows: Pairwise Spearman rank correlations among 15 numeric features and the 1–5 score.

Why it was created: Multivariate analysis to find related features and the strongest numeric correlates of satisfaction (rank correlation suits skewed data).

Insights: The strongest correlate of score is comment length (ρ = -0.377), then delivery days (-0.252), delay (-0.206) and items per order (-0.158). Upper-case ratio is mildly positive (+0.17) – enthusiastic short messages. Price, freight and weight are mutually correlated (heavier products cost more to ship) but nearly uncorrelated with score, so there is little multicollinearity concern for rating-related drivers.

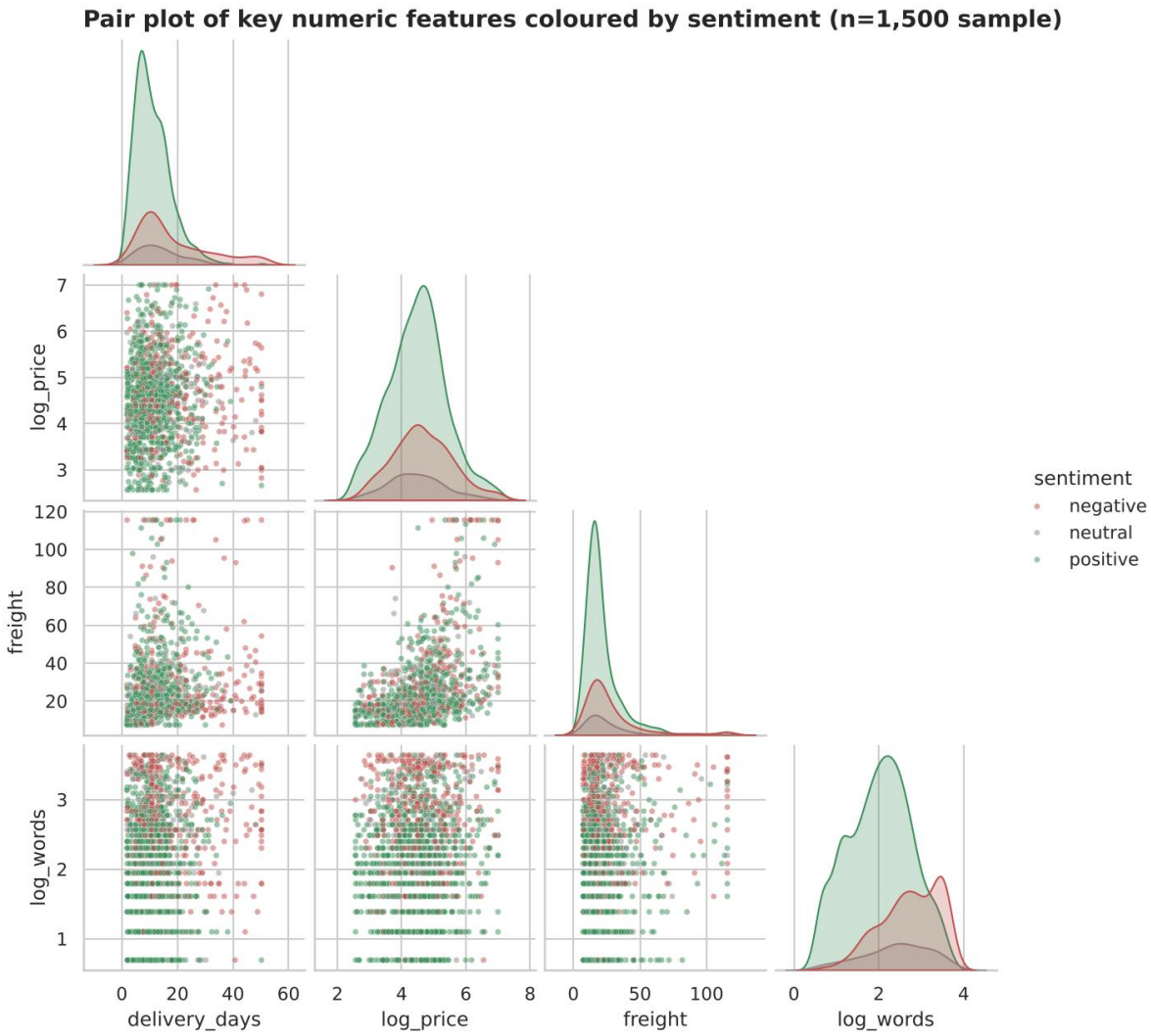


Figure 10. Pair plot of key numeric features coloured by sentiment (1,500-row sample).

What it shows: Scatter plots and density curves of delivery time, log-price, freight and log-comment-length, coloured by sentiment.

Why it was created: To inspect multivariate structure and class separability visually.

Insights: Negative reviews (red) form a longer right tail in delivery_days and sit at higher log_words, but classes overlap heavily on price and freight. No single numeric feature separates the classes, which motivates using the review text itself as the model input. Freight rises with price as expected.

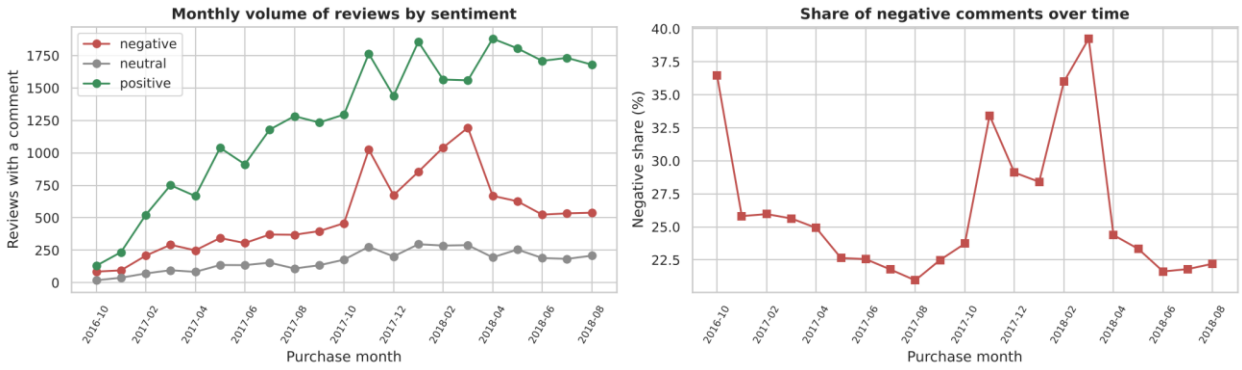


Figure 11. Monthly review volume and negative share over time.

What it shows: Monthly counts of commented reviews by sentiment and the monthly share of negative comments, by purchase month.

Why it was created: To check for time-dependent behaviour such as seasonality or service incidents.

Insights: Volume grows through 2017–2018. The negative share spikes to 33% in Nov 2017 and peaks at 39.3% in 2018-03 (versus ≈22% in mid-2017 and mid-2018). The Nov 2017 peak coincides with a sharp jump in order volume (a holiday-sale surge), which is consistent with delivery capacity being strained, though the data alone cannot prove the cause.

5. Deep Learning Model

5.1 Problem set-up and model choice

The task is three-class sentiment classification of a Portuguese comment. Inputs are the TF-IDF vectors of the cleaned text only: structured fields such as delivery delay are used in the EDA but are not model inputs, because the goal is to understand text and because review_score itself defines the label.

Model: a multi-layer perceptron (deep feed-forward neural network) with ReLU hidden layers and a softmax output, trained by mini-batch Adam with cross-entropy loss. It was compared with a logistic-regression baseline on the same features.

Note on model choice: the assignment asks for the most suitable deep model for text. In the environment used to produce these results, no deep-learning framework (PyTorch/TensorFlow) or pretrained language model could be installed, so a TF-IDF + MLP was the strongest neural model that could be trained and verified end to end. A fine-tuned multilingual or Portuguese transformer (e.g., BERTimbau) is expected to do better, particularly on the neutral class, and is listed under future scope.

5.2 Architecture and hyper-parameters

Item	Setting
Input	TF-IDF 1–2-grams, 17,451 features
Hidden layers	(128,) ReLU units
Output	3-way softmax (negative / neutral / positive)
Trainable parameters	2,234,243
Optimizer / batch size	Adam / 256

Learning rate	0.0003
L2 regularisation (alpha)	0.0001
Max epochs / early stopping	15 / patience 3 on validation macro-F1
Epoch selected	2
Training data	45,914 rows after oversampling (train split only)

5.3 Hyper-parameter tuning

A grid of 2 architectures × 2 learning rates × 2 L2 strengths was searched, selecting by validation macro-F1 (Figure 12).

Hidden layers	Learning rate	L2 alpha	Best epoch	Val macro-F1	Val accuracy
(128,)	0.001	0.0001	1	0.6663	0.8025
(128,)	0.001	0.01	1	0.665	0.8005
(128,)	0.0003	0.0001	2	0.6687	0.813
(128,)	0.0003	0.01	2	0.6677	0.8122
(256, 64)	0.001	0.0001	1	0.648	0.8141
(256, 64)	0.001	0.01	1	0.6501	0.8145
(256, 64)	0.0003	0.0001	1	0.663	0.7933
(256, 64)	0.0003	0.01	1	0.6624	0.7918

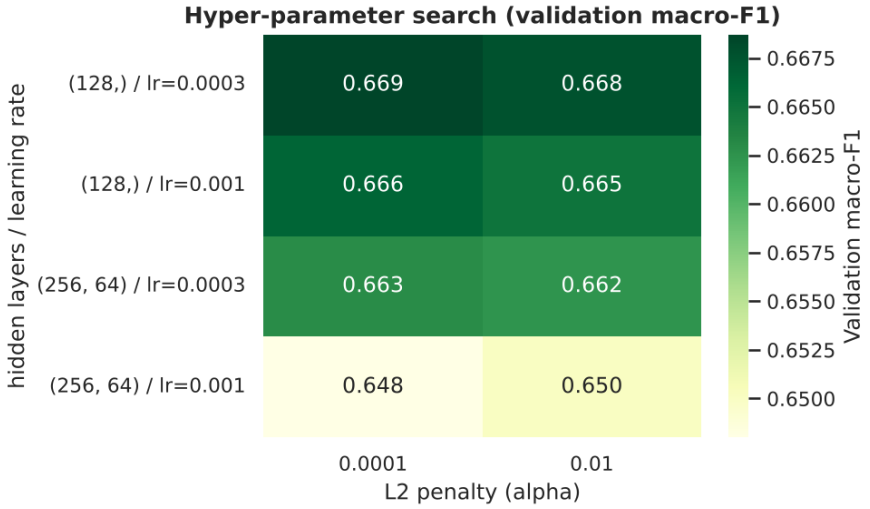


Figure 12. Validation macro-F1 across the hyper-parameter grid.

The single hidden layer of 128 units with a lower learning rate (3e-4) was best (validation macro-F1 0.669). The larger two-layer network reached similar accuracy but a lower macro-F1: with only ≈28k training comments and 17k sparse inputs it over-fits within a single epoch. Regularisation strength made little difference (≤0.002 F1).

5.4 Feature analysis and importance

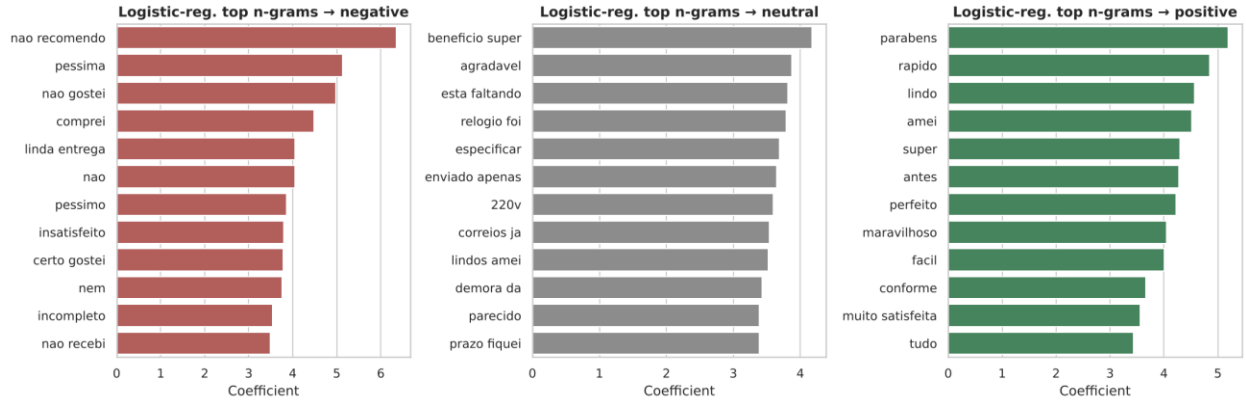


Figure 13. Top n-grams per class according to the logistic-regression coefficients.

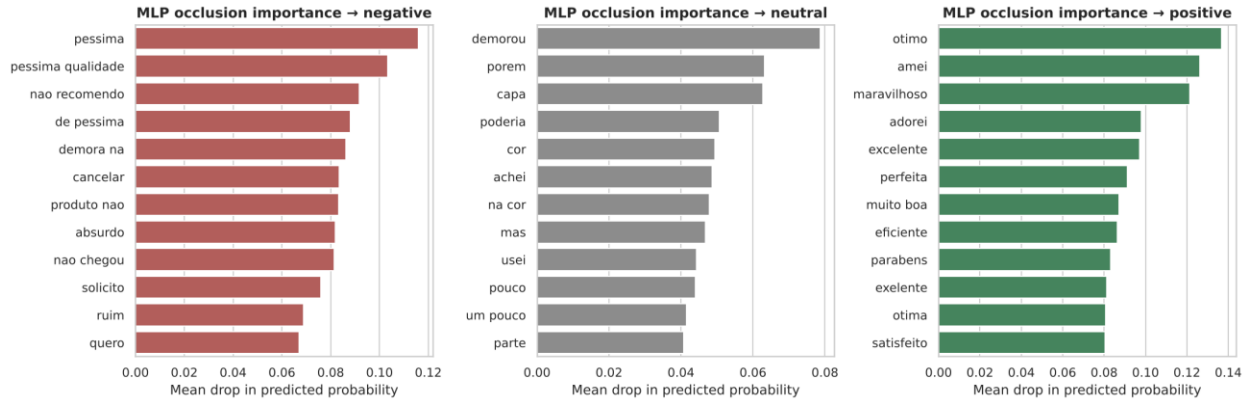


Figure 14. Top features per class by occlusion importance in the MLP.

Logistic-regression coefficients are directly interpretable: 'não recomendo', 'péssima', 'não gostei', 'péssimo', 'insatisfeito' push toward negative, whereas 'parabéns', 'rápido', 'amei', 'perfeito', 'maravilhoso' push toward positive. Neutral coefficients are noisy single product terms, showing that the neutral class has no clear vocabulary of its own. For the MLP, importance was measured by occlusion: each TF-IDF feature in 1,500 test comments was set to zero and the drop in the predicted-class probability averaged. Strong negative drivers are 'péssima (qualidade)', 'não recomendo', 'cancelar', 'absurdo', 'demora na'; positive drivers are 'ótimo', 'amei', 'maravilhoso', 'adorei', 'excelente'. The neutral class is driven by hedging words such as 'porém', 'mas', 'poderia', 'achei', 'demorou' – mixed praise and complaint. This agrees with the EDA: sentiment is encoded in evaluative words and negations, not in structured order data.

6. Result Visualization and Interpretation

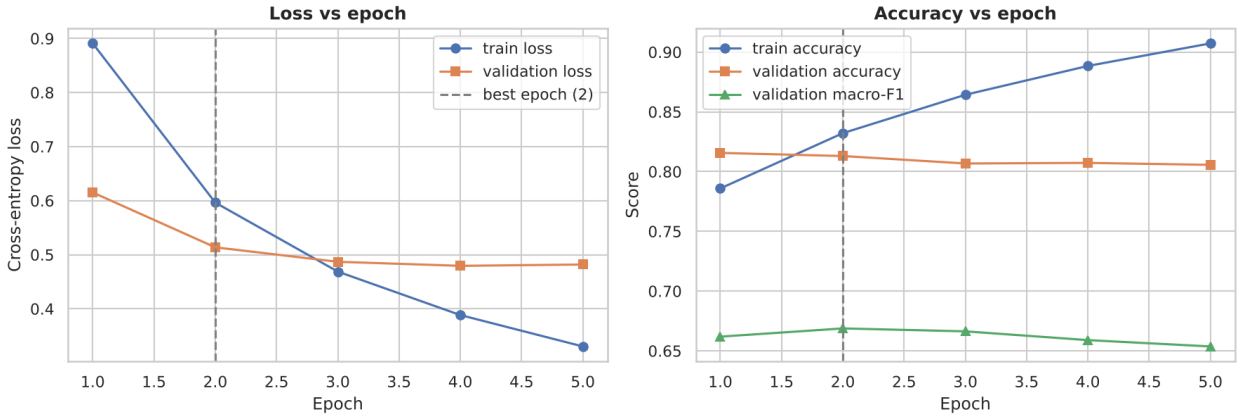


Figure 15. Training/validation loss and accuracy versus epoch.

Training loss falls steadily (0.89 → 0.33) and training accuracy rises to 91%, while validation loss bottoms out near epoch 4 and validation macro-F1 peaks at epoch 2 before declining – the classic sign of over-fitting on sparse text features. Early stopping therefore selected epoch 2, and the model should not be trained longer without stronger regularisation or more data.

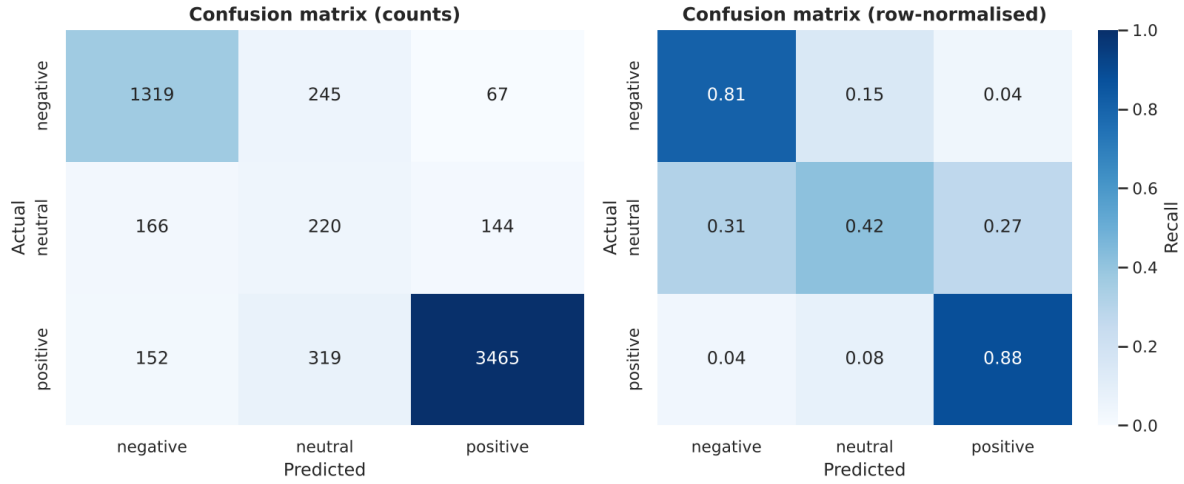


Figure 16. Confusion matrix on the test set (counts and row-normalised).

On 6,097 test reviews, recall is 0.81 for negative, 0.88 for positive and only 0.42 for neutral. Neutral comments are spread over all three classes (31% predicted negative, 27% positive), and 319 positive reviews are predicted neutral. Cross-polarity errors (negative↔positive) are rare (3.6%).

Class	Precision	Recall	F1-score	Support
negative	0.806	0.809	0.807	1631
neutral	0.281	0.415	0.335	530
positive	0.943	0.880	0.910	3936
accuracy			0.821	6097
macro avg	0.676	0.701	0.684	

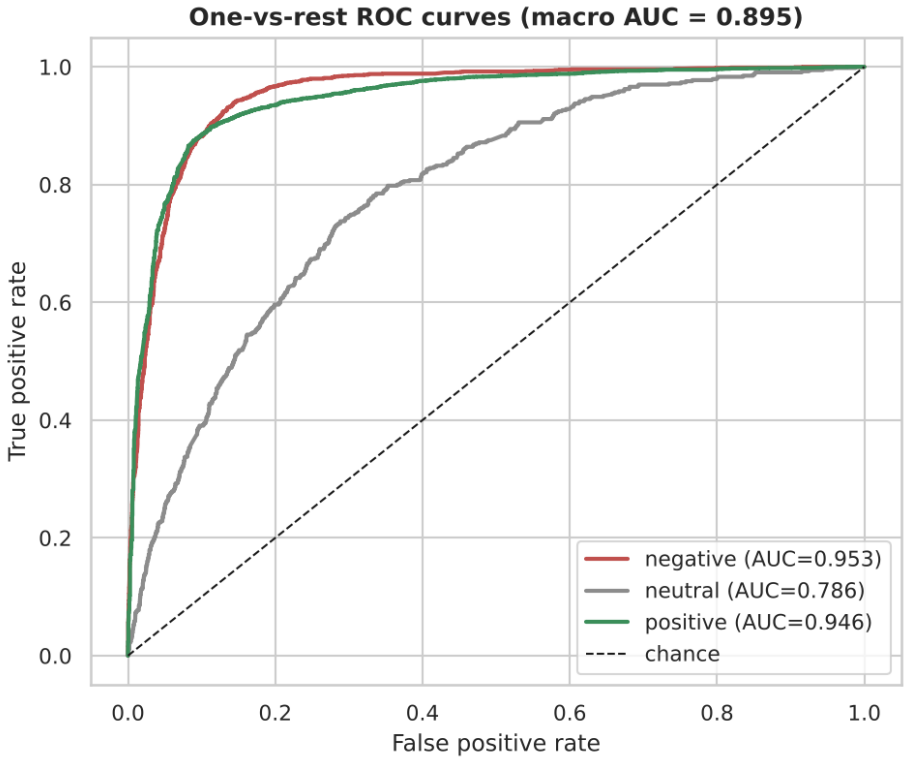


Figure 17. One-vs-rest ROC curves with AUC on the test set.

AUC is 0.953 for negative and 0.946 for positive – the model ranks polar comments very well – but only 0.786 for neutral. Macro one-vs-rest AUC is 0.895 (logistic regression: 0.869).

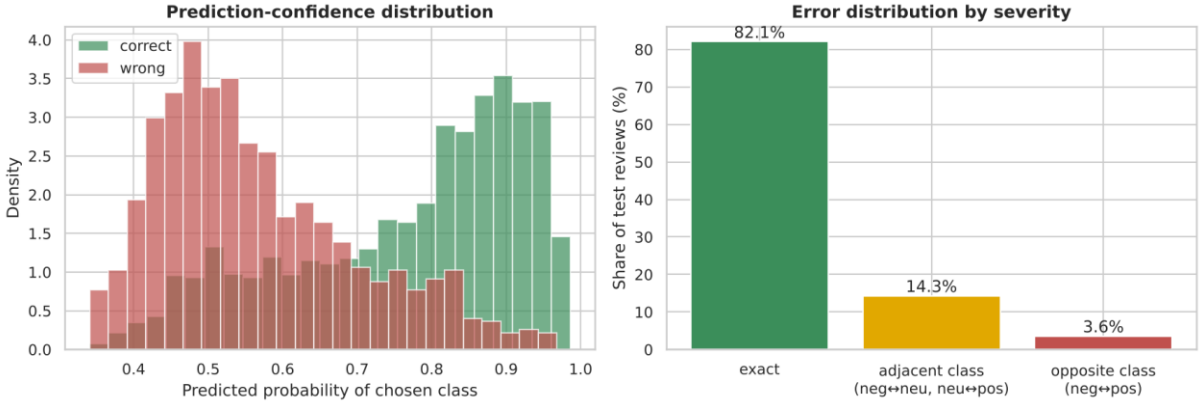


Figure 18. Prediction-confidence distribution and error severity.

Correct predictions have mean confidence 0.77 versus 0.57 for wrong ones, so the probability output is informative: low-confidence predictions can be routed to human review. Of all errors, 14.3% of test cases are adjacent-class mistakes and only 3.6% are opposite-polarity mistakes. (For a classification task, this replaces the regression 'error distribution' chart.)

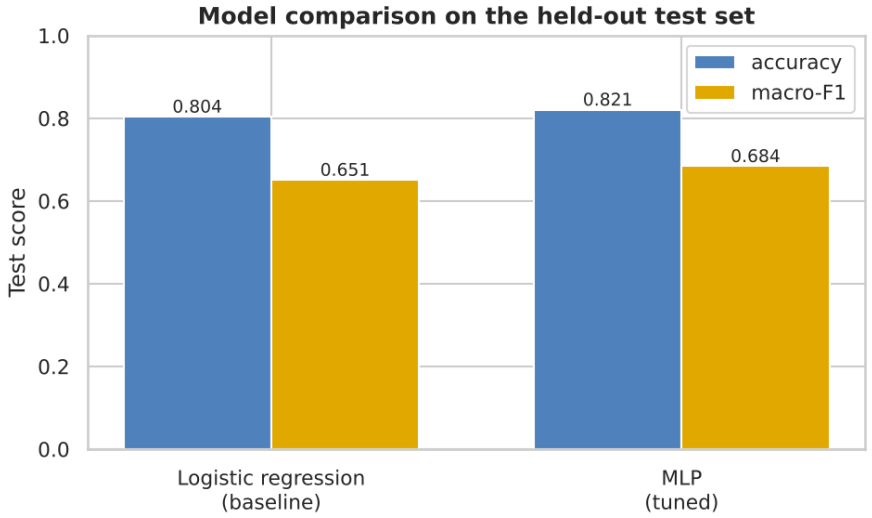


Figure 19. Test accuracy and macro-F1 of the baseline and the tuned MLP.

Model	Val acc	Val macro-F1	Test acc	Test macro-F1	Test macro-AUC
Logistic regression (baseline)	0.801	0.647	0.804	0.651	0.869
MLP (tuned)	0.813	0.669	0.821	0.684	0.895

The MLP improves on the baseline by 1.7 accuracy points and 0.033 macro-F1; validation and test scores are close (0.669 vs 0.684 macro-F1), indicating the result generalises and that tuning did not over-fit the validation set.

7. Conclusion and Future Scope

What was learned

- Customer comments are dominated by delivery topics; 64.56% are positive, 26.75% negative and only 8.69% neutral.
- Late or slow delivery is the clearest operational driver of dissatisfaction (26.37% of negative-review orders were late vs 3.68% of positive).
- Negative reviewers write longer comments and mention defects, invoices, refunds and non-recommendation.
- Category and region effects exist but are small; price and freight barely relate to satisfaction.
- Instalments are over-dispersed (not Poisson); price and comment length are approximately log-normal.

Model performance

The tuned MLP achieved 82.1% accuracy, macro-F1 0.684 and macro AUC 0.895 on unseen data, beating the linear baseline. It is strong on negative (F1 0.81) and positive (F1 0.91) comments but weak on neutral (F1 0.33).

Challenges and limitations

- Strong class imbalance and a vague neutral class (3-star comments often contain both praise and complaint).
- Over-fitting after 2 epochs with sparse TF-IDF inputs; the model has no notion of word order or meaning beyond bigrams.
- Only 40% of reviews contain a comment, so conclusions apply to customers who choose to write; labels come from star ratings, which are themselves noisy.
- No pretrained Portuguese language model or GPU deep-learning framework was available in the environment used for the experiments.
- Correlations are observational and do not prove causes (e.g. delivery delays and the Nov 2017 spike).

Recommendations for the marketplace

- Monitor delivery-promise accuracy: orders predicted to be late are the highest-risk for negative reviews; give honest delivery estimates.
- Route low-confidence or negative predicted comments mentioning 'defeito', 'nota fiscal', 'cancelar' to customer service first.
- Review hardware/furniture categories and high-complaint regions (RJ, MA, CE) with sellers and logistics partners.
- Plan extra logistics capacity ahead of seasonal peaks such as November.

Future scope

- Fine-tune a pretrained Portuguese or multilingual transformer (BERTimbau, XLM-RoBERTa) or a bidirectional LSTM with embeddings; these capture word order and context and should raise neutral-class F1.
- Combine text with structured features (delay, category, price) in a multimodal network and an ordinal (1–5 star) loss.
- Use class-weighted or focal loss and data augmentation for the neutral class; add k-fold cross-validation.
- Add aspect-based sentiment (delivery vs product quality vs seller), topic modelling, and SHAP/attention explanations.
- Deploy as a real-time dashboard that triages incoming reviews.

References

[1] Olist. (2018). Brazilian E-Commerce Public Dataset by Olist [Data set]. Kaggle. https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce

[2] Fry, B. (2008). Visualizing Data: Exploring and Explaining Data with the Processing Environment. O'Reilly Media.

[3] Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

[4] Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. Computing in Science & Engineering, 9(3), 90–95.

[5] Waskom, M. L. (2021). seaborn: statistical data visualization. Journal of Open Source Software, 6(60), 3021.

[6] Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. Proceedings of NAACL-HLT, 4171–4186.

[7] Souza, F., Nogueira, R., & Lotufo, R. (2020). BERTimbau: Pretrained BERT models for Brazilian Portuguese. Intelligent Systems (BRACIS 2020), LNCS 12319, 403–417.

[8] Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. ICLR 2015.

[9] Salton, G., & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. Information Processing & Management, 24(5), 513–523.

Appendix: Code

Complete, commented source (also in the GitHub repository as olist_review_sentiment.py and the .ipynb notebook).

```
# %% [markdown]
# # Exploratory Data Analysis & Visualization of Olist Customer Reviews (Text Data)
# **Course:** U21ADP05 – Exploratory Data Analysis and Visualization | **Assignment II – Case Study**
#
# **Dataset:** Brazilian E-Commerce Public Dataset by Olist (Kaggle: `olistbr/brazilian-ecommerce`).
# Nine relational tables are merged into one review-level table. The *text* field is
# `review_comment_message` (Portuguese customer comments).
#
# **Task:** classify a review comment as *negative* (score 1-2), *neutral* (3) or *positive* (4-5).
#
# Visualization stages (Ben Fry, *Visualizing Data*): 1 Acquire, 2 Parse, 3 Filter, 4 Mine,
# 5 Represent, 6 Refine, 7 Interact. Each is tagged in the cell headers below.
#
# **How to run:** place the Olist CSV files in `./data` (or set `DATA_DIR`) and run all cells
# (or `python olist_review_sentiment.py`). Figures go to `figures/`, numbers to `results/`.


# %% Setup
import os, re, json, copy, time, warnings
from collections import defaultdict
import numpy as np
import pandas as pd
import matplotlib
if "get_ipython" not in globals():
    matplotlib.use("Agg")            # headless when run as a script
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
from scipy import sparse
from sklearn.feature_extraction.text import TfidfVectorizer, CountVectorizer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.neural_network import MLPClassifier
from sklearn.preprocessing import StandardScaler, label_binarize
from sklearn.metrics import (accuracy_score, f1_score, confusion_matrix, roc_curve, auc,
                             roc_auc_score, classification_report)


warnings.filterwarnings("ignore")
SEED = 42
np.random.seed(SEED)
DATA_DIR = os.environ.get("DATA_DIR", "data")
if not os.path.exists(os.path.join(DATA_DIR, "olist_orders_dataset.csv")) and \
        os.path.exists("/kaggle/input/brazilian-ecommerce"):
    DATA_DIR = "/kaggle/input/brazilian-ecommerce"
FIG_DIR, RES_DIR = "figures", "results"
os.makedirs(FIG_DIR, exist_ok=True); os.makedirs(RES_DIR, exist_ok=True)


CLASSES = ["negative", "neutral", "positive"]
PAL = {"negative": "#C0504D", "neutral": "#8C8C8C", "positive": "#3B8E5A"}
sns.set_theme(style="whitegrid", context="notebook", font_scale=0.95)
plt.rcParams.update({"figure.dpi": 110, "axes.titleweight": "bold", "axes.titlesize": 12})


M = {}        # every number the report needs is collected here -> results/metrics.json


def savefig(name):
    plt.tight_layout()
    plt.savefig(f"{FIG_DIR}/{name}.png", dpi=150, bbox_inches="tight")
    plt.show(); plt.close("all")


# %% [markdown]
# ## Stage 1 – ACQUIRE: load the nine Olist tables


# %% Stage 1
def rd(n): return pd.read_csv(os.path.join(DATA_DIR, n))
reviews, orders = rd("olist_order_reviews_dataset.csv"), rd("olist_orders_dataset.csv")
customers, items = rd("olist_customers_dataset.csv"), rd("olist_order_items_dataset.csv")
payments, products = rd("olist_order_payments_dataset.csv"), rd("olist_products_dataset.csv")
sellers, trans = rd("olist_sellers_dataset.csv"), rd("product_category_name_translation.csv")
M["tables"] = {n: list(t.shape) for n, t in dict(reviews=reviews, orders=orders, customers=customers,
               items=items, payments=payments, products=products, sellers=sellers,
               translation=trans).items()}
print(M["tables"])
```

```
# %% [markdown]
# ## Stage 2 – PARSE: merge tables, parse dates, engineer features
# Items and payments are aggregated to one row per order, then everything is left-joined onto the reviews.


# %% Stage 2
for c in ["order_purchase_timestamp", "order_approved_at", "order_delivered_carrier_date",
          "order_delivered_customer_date", "order_estimated_delivery_date"]:
    orders[c] = pd.to_datetime(orders[c])
reviews["review_creation_date"] = pd.to_datetime(reviews["review_creation_date"])


items_agg = items.groupby("order_id").agg(
    n_items=("order_item_id", "count"), total_price=("price", "sum"),
    total_freight=("freight_value", "sum"), n_sellers=("seller_id", "nunique"),
    product_id=("product_id", "first"), seller_id=("seller_id", "first")).reset_index()
pay_agg = payments.groupby("order_id").agg(
    payment_value=("payment_value", "sum"), payment_installments=("payment_installments", "max"),
    payment_type=("payment_type", lambda s: s.mode().iat[0])).reset_index()
products = products.merge(trans, on="product_category_name", how="left")
products["product_volume_cm3"] = products.product_length_cm * products.product_height_cm * products.product_width_cm


df = (reviews.merge(orders, on="order_id", how="left")
      .merge(customers[["customer_id", "customer_state"]], on="customer_id", how="left")
      .merge(items_agg, on="order_id", how="left")
      .merge(pay_agg, on="order_id", how="left")
      .merge(products[["product_id", "product_category_name_english", "product_photos_qty",
                       "product_weight_g", "product_volume_cm3", "product_name_lenght",
                       "product_description_lenght"]], on="product_id", how="left")
      .merge(sellers[["seller_id", "seller_state"]], on="seller_id", how="left"))


# engineered features
df["delivery_days"] = (df.order_delivered_customer_date - df.order_purchase_timestamp).dt.total_seconds() / 86400
df["delay_days"] = (df.order_delivered_customer_date - df.order_estimated_delivery_date).dt.total_seconds() / 86400
df["approval_hours"] = (df.order_approved_at - df.order_purchase_timestamp).dt.total_seconds() / 3600
df["purchase_month"] = df.order_purchase_timestamp.dt.to_period("M").astype(str)
df["purchase_hour"] = df.order_purchase_timestamp.dt.hour
df["has_title"] = df.review_comment_title.notna().astype(int)
M["merged_rows_all_reviews"] = int(len(df))


# keep only reviews that contain a comment (this is the text corpus)
df = df[df.review_comment_message.notna()].copy()
df["text"] = df.review_comment_message.str.replace(r"\s+", " ", regex=True).str.strip()
df["review_char_count"] = df.text.str.len()
df["review_word_count"] = df.text.str.split().str.len()
df["n_exclaim"] = df.text.str.count("!")
df["n_question"] = df.text.str.count(r"\?")
df["upper_ratio"] = df.text.apply(lambda t: sum(c.isupper() for c in t) / max(len(t), 1))
df["sentiment"] = pd.cut(df.review_score, [0, 2, 3, 5], labels=CLASSES).astype(str)
M["rows_with_comment"] = int(len(df))
M["n_columns_merged"] = int(df.shape[1])
print(df.shape); df.head(3)


# %% [markdown]
# ## Stage 3 – FILTER: data understanding & cleaning
# duplicates, missing values, outliers, transformation, encoding, scaling, split.


# %% Stage 3a: missing values + duplicates
miss = (df.isna().mean() * 100).sort_values(ascending=False)
miss = miss[miss > 0]
M["missing_pct_before"] = miss.round(2).to_dict()
fig, ax = plt.subplots(figsize=(8, 5))
sns.barplot(x=miss.values, y=miss.index, color="#4F81BD", ax=ax)
for i, v in enumerate(miss.values): ax.text(v + 0.8, i, f"{v:.1f}%", va="center", fontsize=8)
ax.set_xlabel("Missing values (%)"); ax.set_ylabel(""); ax.set_title("Missing values per field (merged review table)")
ax.set_xlim(0, 105)
savefig("fig01_missing_values")


n0 = len(df)
df = df.drop_duplicates(subset=["order_id", "text"]).copy()          # same review text duplicated for one order
M["duplicates_removed"] = int(n0 - len(df))
df = df[df.review_char_count >= 3].copy()                            # drop empty / 1-2 character comments
M["rows_after_dedup_filter"] = int(len(df))


# %% Stage 3b: imputation
```

```
df["product_category_name_english"] = df.product_category_name_english.fillna("unknown")
df["payment_type"] = df.payment_type.fillna("unknown")
df["delivered"] = df.order_delivered_customer_date.notna().astype(int)
for c in ["delivery_days", "delay_days", "approval_hours", "product_photos_qty", "product_weight_g",
          "product_volume_cm3", "product_name_lenght", "product_description_lenght",
          "total_price", "total_freight", "payment_value", "payment_installments", "n_items"]:
    df[c] = df[c].fillna(df[c].median())                              # median is robust to skew
M["rows_undelivered"] = int((df.delivered == 0).sum())


# %% Stage 3c: outliers (IQR rule) -> winsorise at 1st/99th percentile
NUM = ["total_price", "total_freight", "payment_value", "payment_installments", "n_items",
       "product_photos_qty", "product_weight_g", "product_volume_cm3", "product_name_lenght",
       "product_description_lenght", "delivery_days", "delay_days", "approval_hours",
       "review_word_count", "review_char_count", "n_exclaim", "n_question", "upper_ratio"]
out = {}
for c in NUM:
    q1, q3 = df[c].quantile([.25, .75]); iqr = q3 - q1
    out[c] = int(((df[c] < q1 - 1.5 * iqr) | (df[c] > q3 + 1.5 * iqr)).sum())
M["iqr_outliers"] = out
df_raw_num = df[NUM].copy()                                           # keep unclipped copy for EDA of tails
for c in NUM:
    lo, hi = df[c].quantile([.01, .99]); df[c] = df[c].clip(lo, hi)


# %% Stage 3d: encoding, scaling, split
cat_cols = ["payment_type", "customer_state", "seller_state", "product_category_name_english", "order_status"]
df_enc = pd.get_dummies(df[cat_cols], drop_first=True)
y = df.sentiment.map({c: i for i, c in enumerate(CLASSES)}).values
idx = np.arange(len(df))
tr_idx, tmp_idx = train_test_split(idx, test_size=0.30, stratify=y, random_state=SEED)
va_idx, te_idx = train_test_split(tmp_idx, test_size=0.50, stratify=y[tmp_idx], random_state=SEED)
M["split"] = {"train": len(tr_idx), "val": len(va_idx), "test": len(te_idx)}
scaler = StandardScaler().fit(df.iloc[tr_idx][NUM])                   # fit on TRAIN only (no leakage)
df_scaled = pd.DataFrame(scaler.transform(df[NUM]), columns=NUM, index=df.index)
M["encoded_feature_count"] = int(df_enc.shape[1])
M["class_counts"] = {c: int((df.sentiment == c).sum()) for c in CLASSES}
M["class_pct"] = {c: round(100 * M["class_counts"][c] / len(df), 2) for c in CLASSES}
df.drop(columns=["review_comment_title"]).to_csv(f"{RES_DIR}/clean_review_table.csv", index=False)


# Field list for the report (name, type, short description)
M["describe_num"] = df[NUM].describe().T.round(2).to_dict("index")
print(M["split"], M["class_pct"])


# %% [markdown]
# ## Stage 4 – MINE: statistical tests
# %% Stage 4
S = {}
def cramers_v(ct):
    chi2 = stats.chi2_contingency(ct)[0]; n = ct.values.sum()
    return float(np.sqrt(chi2 / (n * (min(ct.shape) - 1))))
top_cat = df.product_category_name_english.value_counts().index[:15]
for name, col, sub in [("category", "product_category_name_english", df[df.product_category_name_english.isin(top_cat)]),
                       ("customer_state", "customer_state", df), ("payment_type", "payment_type", df)]:
    ct = pd.crosstab(sub[col], sub.sentiment)
    chi2, p, dof, _ = stats.chi2_contingency(ct)
    S[f"chi2_{name}"] = {"chi2": float(chi2), "p": float(p), "dof": int(dof), "cramers_v": cramers_v(ct)}
groups = [df_raw_num.loc[df.sentiment == c, "delay_days"].dropna() for c in CLASSES]
S["kruskal_delay"] = dict(zip(["H", "p"], map(float, stats.kruskal(*groups))))
groups = [df.loc[df.sentiment == c, "review_word_count"] for c in CLASSES]
S["kruskal_wordcount"] = dict(zip(["H", "p"], map(float, stats.kruskal(*groups))))
rho, p = stats.spearmanr(df.delay_days, df.review_score); S["spearman_delay_score"] = {"rho": float(rho), "p": float(p)}
rho, p = stats.spearmanr(df.delivery_days, df.review_score); S["spearman_deliverydays_score"] = {"rho": float(rho), "p":
float(p)}
rho, p = stats.spearmanr(df.review_word_count, df.review_score); S["spearman_words_score"] = {"rho": float(rho), "p":
float(p)}
late = df_raw_num.delay_days > 0
S["late_share_by_sentiment"] = {c: round(float(100 * late[df.sentiment == c].mean()), 2) for c in CLASSES}
S["median_words_by_sentiment"] = {c: float(df.loc[df.sentiment == c, "review_word_count"].median()) for c in CLASSES}
S["mean_delay_by_sentiment"] = {c: round(float(df_raw_num.loc[df.sentiment == c, "delay_days"].mean()), 2) for c in
CLASSES}
S["median_delivery_by_sentiment"] = {c: round(float(df_raw_num.loc[df.sentiment == c, "delivery_days"].median()), 2) for
c in CLASSES}
# discrete distribution: installments vs Poisson ; n_items
inst = df_raw_num.payment_installments.round().astype(int)
S["installments"] = {"mean": float(inst.mean()), "var": float(inst.var()), "dispersion_index": float(inst.var() /
inst.mean())}
# continuous: log-normality of price and review length (skewness before/after log, KS vs fitted normal)
for c in ["total_price", "review_word_count", "delivery_days"]:
    x = df_raw_num[c]; x = x[x > 0]; lx = np.log(x)
```

```
    S[f"dist_{c}"] = {"skew_raw": float(stats.skew(x)), "skew_log": float(stats.skew(lx)),
                      "kurt_raw": float(stats.kurtosis(x)), "ks_lognormal": float(stats.kstest((lx - lx.mean()) /
lx.std(), "norm").statistic)}
M["stats"] = S
print(json.dumps(S, indent=1)[:1500])


# %% [markdown]
# ## Stage 5 & 6 – REPRESENT and REFINE: visualizations
# Every figure has a title, labelled axes, units/legend and a consistent colour code
# (red = negative, grey = neutral, green = positive).


# %% V1 Target distribution
fig, ax = plt.subplots(1, 2, figsize=(11, 4))
sc = df.review_score.value_counts().sort_index()
ax[0].bar(sc.index.astype(str), sc.values, color=["#C0504D", "#C0504D", "#8C8C8C", "#3B8E5A", "#3B8E5A"])
for i, v in enumerate(sc.values): ax[0].text(i, v + 150, f"{v:,}\n({100*v/len(df):.1f}%)", ha="center", fontsize=8)
ax[0].set_xlabel("Review score (stars)"); ax[0].set_ylabel("Number of reviews"); ax[0].set_title("Discrete distribution
of review score"); ax[0].set_ylim(0, sc.max()*1.2)
cc = df.sentiment.value_counts().reindex(CLASSES)
ax[1].pie(cc.values, labels=[f"{c}\n{100*v/len(df):.1f}%" for c, v in cc.items()], colors=[PAL[c] for c in CLASSES],
          startangle=90, wedgeprops={"width": 0.45, "edgecolor": "white"})
ax[1].set_title("Sentiment class balance (target)")
savefig("fig02_target_distribution")


# %% V2 Word frequency + bigrams (text visualisation)
PT_STOP = set("""a o as os um uma uns umas de do da dos das em no na nos nas por para com sem sob sobre e ou mas que se
como
mais menos muito muita muitos muitas já ainda também só até ao aos à às pelo pela pelos pelas num numa isso isto esse
essa esses essas
este esta estes estas aquele aquela eu me meu minha meus minhas você vocês seu sua seus suas nós nosso ele ela eles elas
lhe foi são
ser era eram estou está estão estava estive tem têm tive tinha ter há havia fui vai vou foram pra pro tá to né aí lá aqui
ali
sempre nunca então porque porém pois quando onde qual quais quem tudo todo toda todos todas nada outro outra outros
outras
mesmo mesma bem mal sim não nao produto produtos loja comprei compra recebi recebido""".split())
def clean_viz(t):
    t = re.sub(r"https?://\S+", " ", t.lower()); t = re.sub(r"[^a-záàâãéêíóôõúüç\s]", " ", t)
    return " ".join(w for w in t.split() if len(w) > 2 and w not in PT_STOP)
df["text_viz"] = df.text.apply(clean_viz)
fig, axes = plt.subplots(1, 3, figsize=(15, 4.8), sharex=False)
top_words = {}
for ax, c in zip(axes, CLASSES):
    cv = CountVectorizer(min_df=5); X = cv.fit_transform(df.loc[df.sentiment == c, "text_viz"])
    freq = pd.Series(np.asarray(X.sum(0)).ravel(), index=cv.get_feature_names_out()).sort_values(ascending=False)[:15]
    top_words[c] = freq.index[:10].tolist()
    sns.barplot(x=freq.values, y=freq.index, color=PAL[c], ax=ax)
    ax.set_title(f"Top 15 words – {c}"); ax.set_xlabel("Frequency"); ax.set_ylabel("")
M["top_words"] = top_words
savefig("fig03_word_frequency")


# %% V3 Wordcloud (only if the wordcloud package is installed; Colab/Kaggle have it) + N-grams
try:
    from wordcloud import WordCloud
    fig, axes = plt.subplots(1, 3, figsize=(15, 4.5))
    for ax, c in zip(axes, CLASSES):
        wc = WordCloud(width=600, height=400, background_color="white", colormap="Greens" if c == "positive" else "Reds"
if c == "negative" else "Greys",
                       random_state=SEED).generate(" ".join(df.loc[df.sentiment == c, "text_viz"]))
        ax.imshow(wc, interpolation="bilinear"); ax.axis("off"); ax.set_title(f"Word cloud – {c}")
    savefig("fig03b_wordclouds")
except ImportError:
    print("wordcloud not installed -> skipped (frequency plots above cover the same information)")


fig, axes = plt.subplots(1, 3, figsize=(15, 4.8))
bigrams = {}
for ax, c in zip(axes, CLASSES):
    cv = CountVectorizer(ngram_range=(2, 2), min_df=5); X = cv.fit_transform(df.loc[df.sentiment == c, "text_viz"])
    freq = pd.Series(np.asarray(X.sum(0)).ravel(), index=cv.get_feature_names_out()).sort_values(ascending=False)[:12]
    bigrams[c] = freq.index[:8].tolist()
    sns.barplot(x=freq.values, y=freq.index, color=PAL[c], ax=ax)
    ax.set_title(f"Top bigrams – {c}"); ax.set_xlabel("Frequency"); ax.set_ylabel("")
M["top_bigrams"] = bigrams
savefig("fig04_ngrams")


# %% V4 Review length: univariate distribution + by sentiment
```

```
fig, ax = plt.subplots(1, 3, figsize=(16, 4.3))
wc_ = df_raw_num.review_word_count
ax[0].hist(wc_[wc_ < wc_.quantile(.995)], bins=50, color="#4F81BD", edgecolor="white", density=True)
ax[0].set_xlabel("Words per review"); ax[0].set_ylabel("Density"); ax[0].set_title("Review length (right-skewed)")
lx = np.log(wc_[wc_ > 0]); xs = np.linspace(lx.min(), lx.max(), 200)
ax[1].hist(lx, bins=40, density=True, color="#4F81BD", edgecolor="white", label="log(words)")
ax[1].plot(xs, stats.norm.pdf(xs, lx.mean(), lx.std()), "r-", lw=2, label="fitted normal"); ax[1].legend()
ax[1].set_xlabel("log(words per review)"); ax[1].set_ylabel("Density"); ax[1].set_title("Log-transform ≈ normal (log-
normal length)")
sns.boxplot(data=df_raw_num.assign(sentiment=df.sentiment.values), x="sentiment", y="review_word_count", order=CLASSES,
            palette=PAL, showfliers=False, ax=ax[2])
ax[2].set_xlabel("Sentiment"); ax[2].set_ylabel("Words per review"); ax[2].set_title("Review length by sentiment
(outliers hidden)")
savefig("fig05_review_length")


# %% V5 Delivery vs sentiment (bivariate)
d = df_raw_num.assign(sentiment=df.sentiment.values, score=df.review_score.values)
fig, ax = plt.subplots(1, 3, figsize=(16, 4.3))
sns.boxplot(data=d[d.delay_days.between(-40, 40)], x="sentiment", y="delay_days", order=CLASSES, palette=PAL,
showfliers=False, ax=ax[0])
ax[0].axhline(0, color="k", ls="--", lw=1); ax[0].set_ylabel("Delivery delay vs estimate (days; >0 = late)");
ax[0].set_xlabel("Sentiment"); ax[0].set_title("Late delivery ⇔ negative reviews")
lr = d.assign(late=d.delay_days > 0).groupby("score").late.mean() * 100
ax[1].bar(lr.index.astype(str), lr.values, color=["#C0504D", "#C0504D", "#8C8C8C", "#3B8E5A", "#3B8E5A"])
for i, v in enumerate(lr.values): ax[1].text(i, v + .4, f"{v:.1f}%", ha="center", fontsize=9)
ax[1].set_xlabel("Review score"); ax[1].set_ylabel("Orders delivered late (%)"); ax[1].set_title("Share of late
deliveries by score")
sns.violinplot(data=d[d.delivery_days < 60], x="sentiment", y="delivery_days", order=CLASSES, palette=PAL, cut=0,
ax=ax[2])
ax[2].set_xlabel("Sentiment"); ax[2].set_ylabel("Delivery time (days)"); ax[2].set_title("Delivery time distribution by
sentiment")
savefig("fig06_delivery_vs_sentiment")


# %% V6 Category and state view (categorical bivariate)
cat = pd.crosstab(df.product_category_name_english, df.sentiment, normalize="index") * 100
vol = df.product_category_name_english.value_counts()
top = vol.index[:12]; cat = cat.loc[top].sort_values("negative")
st = pd.crosstab(df.customer_state, df.sentiment, normalize="index") * 100
st = st[df.customer_state.value_counts().reindex(st.index) > 300].sort_values("negative")
fig, ax = plt.subplots(1, 2, figsize=(15, 5.2))
cat[CLASSES].plot(kind="barh", stacked=True, color=[PAL[c] for c in CLASSES], ax=ax[0], width=.8)
ax[0].set_xlabel("% of reviews"); ax[0].set_ylabel(""); ax[0].set_title("Sentiment mix – 12 biggest product categories");
ax[0].legend(loc="upper center", bbox_to_anchor=(.5, -.12), ncol=3, fontsize=8)
st[CLASSES].plot(kind="barh", stacked=True, color=[PAL[c] for c in CLASSES], ax=ax[1], width=.8)
ax[1].set_xlabel("% of reviews"); ax[1].set_ylabel(""); ax[1].set_title("Sentiment mix – customer states (>300
reviews)"); ax[1].legend(loc="upper center", bbox_to_anchor=(.5, -.12), ncol=3, fontsize=8)
M["category_neg_pct"] = cat.negative.round(1).to_dict(); M["state_neg_pct"] = st.negative.round(1).to_dict()
savefig("fig07_category_state_sentiment")


# %% V7 Discrete & continuous distributions
fig, ax = plt.subplots(2, 2, figsize=(12, 8.5))
k = np.arange(0, 13); obs = inst.value_counts(normalize=True).reindex(k, fill_value=0)
ax[0, 0].bar(k, obs.values, color="#4F81BD", label="observed")
ax[0, 0].plot(k, stats.poisson.pmf(k, inst.mean()), "ro-", label=f"Poisson(λ={inst.mean():.2f})")
ax[0, 0].set_xlabel("Payment installments"); ax[0, 0].set_ylabel("Probability"); ax[0, 0].set_title("Discrete:
installments vs Poisson"); ax[0, 0].legend()
ni = df_raw_num.n_items.round().astype(int).value_counts(normalize=True).sort_index()[:8]
ax[0, 1].bar(ni.index, ni.values, color="#4F81BD"); ax[0, 1].set_yscale("log")
ax[0, 1].set_xlabel("Items per order"); ax[0, 1].set_ylabel("Probability (log scale)"); ax[0, 1].set_title("Discrete:
items per order (geometric-like decay)")
lp = np.log(df_raw_num.total_price[df_raw_num.total_price > 0]); xs = np.linspace(lp.min(), lp.max(), 200)
ax[1, 0].hist(lp, bins=50, density=True, color="#4F81BD", edgecolor="white", label="log(price)")
ax[1, 0].plot(xs, stats.norm.pdf(xs, lp.mean(), lp.std()), "r-", lw=2, label="fitted normal"); ax[1, 0].legend()
ax[1, 0].set_xlabel("log(order price, BRL)"); ax[1, 0].set_ylabel("Density"); ax[1, 0].set_title("Continuous: price is
approx. log-normal")
stats.probplot(lp, dist="norm", plot=ax[1, 1]); ax[1, 1].get_lines()[0].set_markersize(2)
ax[1, 1].set_title("Q-Q plot of log(price) vs normal"); ax[1, 1].set_xlabel("Theoretical quantiles"); ax[1,
1].set_ylabel("Ordered values")
savefig("fig08_distributions")


# %% V8 Correlation heatmap
corr_cols = ["review_score", "delivery_days", "delay_days", "approval_hours", "total_price", "total_freight",
"payment_value",
             "payment_installments", "n_items", "product_photos_qty", "product_weight_g", "product_volume_cm3",
             "product_description_lenght", "review_word_count", "n_exclaim", "upper_ratio"]
cm = df[corr_cols].corr(method="spearman")
fig, ax = plt.subplots(figsize=(10.5, 8.5))
```

```
sns.heatmap(cm, annot=True, fmt=".2f", cmap="RdBu_r", center=0, vmin=-1, vmax=1, annot_kws={"size": 7},
cbar_kws={"label": "Spearman ρ"}, ax=ax)
ax.set_title("Spearman correlation of numeric features and review score")
savefig("fig09_correlation_heatmap")
M["corr_with_score"] = cm["review_score"].drop("review_score").round(3).sort_values().to_dict()


# %% V9 Pair plot
samp = df.sample(1500, random_state=SEED)
pp = pd.DataFrame({"delivery_days": samp.delivery_days, "log_price": np.log1p(samp.total_price),
                   "freight": samp.total_freight, "log_words": np.log1p(samp.review_word_count), "sentiment":
samp.sentiment})
g = sns.pairplot(pp, hue="sentiment", hue_order=CLASSES, palette=PAL, corner=True, plot_kws={"s": 12, "alpha": .5},
height=2.1)
g.figure.suptitle("Pair plot of key numeric features coloured by sentiment (n=1,500 sample)", y=1.01, fontweight="bold")
g.figure.savefig(f"{FIG_DIR}/fig10_pairplot.png", dpi=150, bbox_inches="tight"); plt.show(); plt.close("all")


# %% V10 Time trend
tm = pd.crosstab(df.purchase_month, df.sentiment)[CLASSES]; tm = tm[tm.sum(1) > 100]
fig, ax = plt.subplots(1, 2, figsize=(14, 4.3))
for c in CLASSES: ax[0].plot(tm.index, tm[c], marker="o", color=PAL[c], label=c)
ax[0].set_xticks(range(0, len(tm), 2)); ax[0].set_xticklabels(tm.index[::2], rotation=60, fontsize=8)
ax[0].set_xlabel("Purchase month"); ax[0].set_ylabel("Reviews with a comment"); ax[0].set_title("Monthly volume of
reviews by sentiment"); ax[0].legend()
neg_share = tm.negative / tm.sum(1) * 100
ax[1].plot(neg_share.index, neg_share.values, marker="s", color=PAL["negative"])
ax[1].set_xticks(range(0, len(tm), 2)); ax[1].set_xticklabels(tm.index[::2], rotation=60, fontsize=8)
ax[1].set_xlabel("Purchase month"); ax[1].set_ylabel("Negative share (%)"); ax[1].set_title("Share of negative comments
over time")
M["peak_neg_month"] = [neg_share.idxmax(), round(float(neg_share.max()), 1)]
savefig("fig11_time_trend")


# %% Stage 7 – INTERACT: optional interactive Plotly chart (runs only if plotly is installed)
try:
    import plotly.express as px
    fig = px.scatter(df.sample(3000, random_state=SEED), x="delivery_days", y="review_word_count", color="sentiment",
                     color_discrete_map=PAL, hover_data=["product_category_name_english", "review_score",
"customer_state"],
                     log_y=True, title="Interactive: delivery time vs review length (hover for details)")
    fig.write_html(f"{FIG_DIR}/interactive_delivery_vs_length.html")
except ImportError:
    print("plotly not installed -> interactive chart skipped")


# %% [markdown]
# ## Step 4 – MODEL BUILDING
# **Features for the model:** the review text only (TF-IDF of 1–2-grams). Structured fields such as delivery delay are
# analysed in the EDA but not fed to the model: this keeps the task a pure text-understanding problem and avoids using
# `review_score` (the label source) or post-purchase fields that would not exist when only a comment is available.
#
# **Model:** Multi-Layer Perceptron (deep feed-forward neural network) trained with Adam, mini-batches and per-epoch
# monitoring, compared with a Logistic-Regression baseline.


# %% Text prep + TF-IDF
def clean_model(t): return re.sub(r"https?://\S+", " ", t.lower())
texts = df.text.apply(clean_model).values
vec = TfidfVectorizer(ngram_range=(1, 2), min_df=3, max_features=20000, sublinear_tf=True, strip_accents="unicode")
Xtr = vec.fit_transform(texts[tr_idx]); Xva = vec.transform(texts[va_idx]); Xte = vec.transform(texts[te_idx])
ytr, yva, yte = y[tr_idx], y[va_idx], y[te_idx]
fn = np.array(vec.get_feature_names_out())
M["tfidf_vocab_size"] = int(len(fn))
print(Xtr.shape, Xva.shape, Xte.shape)


# class-balance the TRAIN set only (random oversampling of minority classes up to 75% of majority)
rng = np.random.RandomState(SEED)
cnt = np.bincount(ytr); target = int(.75 * cnt.max()); parts = []
for k in range(3):
    ids = np.where(ytr == k)[0]
    if len(ids) < target: ids = np.concatenate([ids, rng.choice(ids, target - len(ids), replace=True)])
    parts.append(ids)
bal = rng.permutation(np.concatenate(parts)); Xtr_b, ytr_b = Xtr[bal], ytr[bal]
M["train_after_oversampling"] = int(len(bal))


# %% Baseline: logistic regression (also gives interpretable feature importance)
lr_clf = LogisticRegression(C=4, max_iter=1000, random_state=SEED).fit(Xtr_b, ytr_b)
M["baseline_lr"] = {"val_acc": float(accuracy_score(yva, lr_clf.predict(Xva))),
                    "val_f1_macro": float(f1_score(yva, lr_clf.predict(Xva), average="macro")),
                    "test_acc": float(accuracy_score(yte, lr_clf.predict(Xte))),
```

```
                  "test_f1_macro": float(f1_score(yte, lr_clf.predict(Xte), average="macro"))}
print(M["baseline_lr"])


# %% MLP training with epoch-level tracking + basic hyper-parameter tuning
def train_mlp(hidden, lr, alpha, max_epochs=15, patience=3, verbose=False):
    clf = MLPClassifier(hidden_layer_sizes=hidden, solver="adam", learning_rate_init=lr, alpha=alpha,
                        batch_size=256, random_state=SEED)
    hist = {"train_loss": [], "val_loss": [], "train_acc": [], "val_acc": [], "val_f1": []}
    best, best_f1, bad = None, -1, 0
    for ep in range(max_epochs):
        clf.partial_fit(Xtr_b, ytr_b, classes=[0, 1, 2])
        pv = clf.predict_proba(Xva)
        hist["train_loss"].append(clf.loss_)
        hist["val_loss"].append(float(-np.mean(np.log(np.clip(pv[np.arange(len(yva)), yva], 1e-12, 1)))))
        hist["train_acc"].append(float(accuracy_score(ytr_b[:8000], clf.predict(Xtr_b[:8000]))))   # 8k random train rows
(rows are shuffled)
        hist["val_acc"].append(float(accuracy_score(yva, pv.argmax(1))))
        f1 = float(f1_score(yva, pv.argmax(1), average="macro")); hist["val_f1"].append(f1)
        if f1 > best_f1 + 1e-4: best_f1, best, bad, best_ep = f1, copy.deepcopy(clf), 0, ep + 1
        else:
            bad += 1
            if bad >= patience: break
    return best, hist, best_f1, best_ep


grid = [(h, lr, a) for h in [(128,), (256, 64)] for lr in [1e-3, 3e-4] for a in [1e-4, 1e-2]]
rows, best_all = [], None
t0 = time.time()
for h, lr, a in grid:
    m, hist, f1, bep = train_mlp(h, lr, a)
    rows.append({"hidden_layers": str(h), "learning_rate": lr, "alpha_L2": a, "best_epoch": bep,
                 "val_macro_f1": round(f1, 4), "val_acc": round(hist["val_acc"][bep - 1], 4)})
    print(rows[-1], f"{time.time()-t0:.0f}s")
    if best_all is None or f1 > best_all[2]: best_all = (m, hist, f1, (h, lr, a), bep)
hp = pd.DataFrame(rows); hp.to_csv(f"{RES_DIR}/hyperparameter_search.csv", index=False)
mlp, hist, _, best_cfg, best_ep = best_all
M["hp_grid"] = rows
M["best_config"] = {"hidden_layers": str(best_cfg[0]), "learning_rate": best_cfg[1], "alpha_L2": best_cfg[2],
"best_epoch": best_ep,
                    "batch_size": 256, "optimizer": "Adam", "activation": "ReLU", "max_epochs": 15,
"early_stopping_patience": 3}
M["history"] = hist
M["n_parameters"] = int(sum(w.size for w in mlp.coefs_) + sum(b.size for b in mlp.intercepts_))
print(M["best_config"], M["n_parameters"])


# %% Hyper-parameter heat-map
pv_ = hp.assign(cfg=hp.hidden_layers + " / lr=" + hp.learning_rate.astype(str)).pivot(index="cfg", columns="alpha_L2",
values="val_macro_f1")
fig, ax = plt.subplots(figsize=(6.5, 4))
sns.heatmap(pv_, annot=True, fmt=".3f", cmap="YlGn", cbar_kws={"label": "Validation macro-F1"}, ax=ax)
ax.set_xlabel("L2 penalty (alpha)"); ax.set_ylabel("hidden layers / learning rate"); ax.set_title("Hyper-parameter search
(validation macro-F1)")
savefig("fig12_hyperparameter_search")


# %% Feature analysis: (a) LR coefficients, (b) MLP occlusion importance
coef = lr_clf.coef_
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
M["lr_top_features"] = {}
for k, ax in enumerate(axes):
    o = np.argsort(coef[k])[::-1][:12]
    sns.barplot(x=coef[k][o], y=fn[o], color=PAL[CLASSES[k]], ax=ax)
    ax.set_title(f"Logistic-reg. top n-grams → {CLASSES[k]}"); ax.set_xlabel("Coefficient"); ax.set_ylabel("")
    M["lr_top_features"][CLASSES[k]] = fn[o][:8].tolist()
savefig("fig13_lr_feature_importance")


def occlusion(model, X, n_docs=1500):
    """Zero one TF-IDF feature at a time and record the drop in the predicted-class probability."""
    sel = np.random.RandomState(SEED).choice(X.shape[0], min(n_docs, X.shape[0]), replace=False)
    Xs = X[sel].tocsr(); p0 = model.predict_proba(Xs); cls = p0.argmax(1)
    data, ind, ptr, owner = [], [], [0], []
    for i in range(Xs.shape[0]):
        r = Xs[i]
        for j in range(len(r.indices)):
            data.append(np.delete(r.data, j)); ind.append(np.delete(r.indices, j)); ptr.append(ptr[-1] + len(r.data) - 1)
            owner.append((i, r.indices[j]))
    V = sparse.csr_matrix((np.concatenate(data), np.concatenate(ind), ptr), shape=(len(owner), X.shape[1]))
    pm = model.predict_proba(V); agg = defaultdict(list)
    for (i, j), p in zip(owner, pm):
```

```
        agg[(cls[i], j)].append(p0[i, cls[i]] - p[cls[i]])
    return agg
agg = occlusion(mlp, Xte)
fig, axes = plt.subplots(1, 3, figsize=(15, 5)); M["mlp_top_features"] = {}
for k, ax in enumerate(axes):
    r = [(fn[j], np.mean(v), len(v)) for (c, j), v in agg.items() if c == k and len(v) >= 5]
    r = sorted(r, key=lambda t: -t[1])[:12]
    sns.barplot(x=[t[1] for t in r], y=[t[0] for t in r], color=PAL[CLASSES[k]], ax=ax)
    ax.set_title(f"MLP occlusion importance → {CLASSES[k]}"); ax.set_xlabel("Mean drop in predicted probability");
ax.set_ylabel("")
    M["mlp_top_features"][CLASSES[k]] = [t[0] for t in r[:8]]
savefig("fig14_mlp_feature_importance")


# %% [markdown]
# ## Step 5 – MODEL EVALUATION VISUALIZATION


# %% Loss & accuracy curves
ep = np.arange(1, len(hist["train_loss"]) + 1)
fig, ax = plt.subplots(1, 2, figsize=(12, 4.2))
ax[0].plot(ep, hist["train_loss"], "o-", label="train loss"); ax[0].plot(ep, hist["val_loss"], "s-", label="validation
loss")
ax[0].axvline(best_ep, color="grey", ls="--", label=f"best epoch ({best_ep})")
ax[0].set_xlabel("Epoch"); ax[0].set_ylabel("Cross-entropy loss"); ax[0].set_title("Loss vs epoch"); ax[0].legend()
ax[1].plot(ep, hist["train_acc"], "o-", label="train accuracy"); ax[1].plot(ep, hist["val_acc"], "s-", label="validation
accuracy")
ax[1].plot(ep, hist["val_f1"], "^-", label="validation macro-F1"); ax[1].axvline(best_ep, color="grey", ls="--")
ax[1].set_xlabel("Epoch"); ax[1].set_ylabel("Score"); ax[1].set_title("Accuracy vs epoch"); ax[1].legend()
savefig("fig15_loss_accuracy_curves")


# %% Test-set evaluation: confusion matrix, ROC/AUC, confidence
P = mlp.predict_proba(Xte); pred = P.argmax(1)
M["mlp_test"] = {"acc": float(accuracy_score(yte, pred)), "f1_macro": float(f1_score(yte, pred, average="macro")),
                 "f1_weighted": float(f1_score(yte, pred, average="weighted")),
                 "report": classification_report(yte, pred, target_names=CLASSES, output_dict=True)}
M["mlp_val"] = {"acc": hist["val_acc"][best_ep - 1], "f1_macro": hist["val_f1"][best_ep - 1]}
cmx = confusion_matrix(yte, pred)
fig, ax = plt.subplots(1, 2, figsize=(11, 4.6))
sns.heatmap(cmx, annot=True, fmt="d", cmap="Blues", xticklabels=CLASSES, yticklabels=CLASSES, cbar=False, ax=ax[0])
ax[0].set_xlabel("Predicted"); ax[0].set_ylabel("Actual"); ax[0].set_title("Confusion matrix (counts)")
sns.heatmap(cmx / cmx.sum(1, keepdims=True), annot=True, fmt=".2f", cmap="Blues", vmin=0, vmax=1, xticklabels=CLASSES,
yticklabels=CLASSES,
            cbar_kws={"label": "Recall"}, ax=ax[1])
ax[1].set_xlabel("Predicted"); ax[1].set_ylabel("Actual"); ax[1].set_title("Confusion matrix (row-normalised)")
M["confusion_matrix"] = cmx.tolist()
savefig("fig16_confusion_matrix")


Yb = label_binarize(yte, classes=[0, 1, 2]); fig, ax = plt.subplots(figsize=(6.5, 5.5)); M["auc"] = {}
for k in range(3):
    f_, t_, _ = roc_curve(Yb[:, k], P[:, k]); M["auc"][CLASSES[k]] = float(auc(f_, t_))
    ax.plot(f_, t_, color=PAL[CLASSES[k]], lw=2, label=f"{CLASSES[k]} (AUC={M['auc'][CLASSES[k]]:.3f})")
M["auc"]["macro_ovr"] = float(roc_auc_score(yte, P, multi_class="ovr", average="macro"))
M["auc_lr_macro_ovr"] = float(roc_auc_score(yte, lr_clf.predict_proba(Xte), multi_class="ovr", average="macro"))
ax.plot([0, 1], [0, 1], "k--", lw=1, label="chance"); ax.set_xlabel("False positive rate"); ax.set_ylabel("True positive
rate")
ax.set_title(f"One-vs-rest ROC curves (macro AUC = {M['auc']['macro_ovr']:.3f})"); ax.legend(loc="lower right")
savefig("fig17_roc_curves")


conf = P.max(1); ok = pred == yte
fig, ax = plt.subplots(1, 2, figsize=(12, 4.2))
ax[0].hist(conf[ok], bins=25, alpha=.7, color="#3B8E5A", label="correct", density=True)
ax[0].hist(conf[~ok], bins=25, alpha=.7, color="#C0504D", label="wrong", density=True)
ax[0].set_xlabel("Predicted probability of chosen class"); ax[0].set_ylabel("Density"); ax[0].set_title("Prediction-
confidence distribution"); ax[0].legend()
gap = np.abs(pred - yte); M["error_distance_pct"] = {"exact": float(100 * (gap == 0).mean()), "off_by_one_class":
float(100 * (gap == 1).mean()),
                                                    "negative_vs_positive_confusion": float(100 * (gap == 2).mean())}
ax[1].bar(["exact", "adjacent class\n(neg↔neu, neu↔pos)", "opposite class\n(neg↔pos)"], [100 * (gap == 0).mean(), 100 *
(gap == 1).mean(), 100 * (gap == 2).mean()],
          color=["#3B8E5A", "#E1A800", "#C0504D"])
for i, v in enumerate(M["error_distance_pct"].values()): ax[1].text(i, v + 1, f"{v:.1f}%", ha="center")
ax[1].set_ylabel("Share of test reviews (%)"); ax[1].set_title("Error distribution by severity")
M["confidence"] = {"mean_correct": float(conf[ok].mean()), "mean_wrong": float(conf[~ok].mean())}
savefig("fig18_error_analysis")


fig, ax = plt.subplots(figsize=(6.5, 4))
mods = ["Logistic regression\n(baseline)", "MLP\n(tuned)"]
```

```
a = [M["baseline_lr"]["test_acc"], M["mlp_test"]["acc"]]; f = [M["baseline_lr"]["test_f1_macro"],
M["mlp_test"]["f1_macro"]]
x = np.arange(2); ax.bar(x - .18, a, .36, label="accuracy", color="#4F81BD"); ax.bar(x + .18, f, .36, label="macro-F1",
color="#E1A800")
for i in range(2): ax.text(i - .18, a[i] + .01, f"{a[i]:.3f}", ha="center", fontsize=9); ax.text(i + .18, f[i] + .01,
f"{f[i]:.3f}", ha="center", fontsize=9)
ax.set_xticks(x); ax.set_xticklabels(mods); ax.set_ylim(0, 1); ax.set_ylabel("Test score"); ax.set_title("Model
comparison on the held-out test set"); ax.legend()
savefig("fig19_model_comparison")


# %% Save everything
def _j(o):
    if isinstance(o, (np.integer,)): return int(o)
    if isinstance(o, (np.floating,)): return float(o)
    return str(o)
json.dump(M, open(f"{RES_DIR}/metrics.json", "w"), indent=1, default=_j)
print("Done. Test accuracy %.3f | macro-F1 %.3f | macro AUC %.3f" % (M["mlp_test"]["acc"], M["mlp_test"]["f1_macro"],
M["auc"]["macro_ovr"]))
```